# pyaces 개발자 매뉴얼

ACES II 를 Python 에서 부르는 f2py 확장 모듈 `aces2py` 의 구조 · 빌드 · 확장 · 테스트

| | |
|---|---|
| **저장소** | `ACES_PROGRAM/pyaces` (GitHub youngchoonpark/ACES, private) |
| **기준 커밋** | `eab3c4e6` (master, 2026-10-14) |
| **작성일** | 2026-10-23 |
| **검증 상태** | pyaces 스위트 51/51 PASS — 로컬 워크스테이션(oneAPI 2024.0) · ccsd 서버(oneAPI 2023.2) |
| **링크 감사** | collision baseline 296 항목, COMMON shadow 0 — 두 기계에서 동일 |

이 문서는 소스 코드(`pyaces/build.sh`, `a2_pyf/*.F`, 생성된 `Main/gen/aces2pymodule.c`, 테스트 스크립트)와 빌드된 `.so` 를 직접 읽고 확인한 내용으로 작성했다. 줄 번호는 기준 커밋 기준이다.

# 목차

1. 개요
2. 디렉터리 구조
3. 빌드 환경
4. 빌드 파이프라인 (`build.sh`)
5. 하나의 .so 로 링크하기 — 심볼 충돌과 해결
6. 링크 결과 감사 도구 (`collision_audit.py`)
7. `aces2py` 모듈 API
8. 계산 실행하기
9. 테스트 스위트
10. 확장하기 — 새 진입점과 새 테스트
11. 알려진 문제 (수정 내역 포함)
12. 개발 이력과 서버 배포
부록 A. 레거시 파일 · 부록 B. 명령 요약

# 1. 개요

**pyaces** 는 Fortran 양자화학 프로그램 ACES II(`ACES_PROGRAM/aces2`)의 계산 모듈을 Python 에서 직접 호출하기 위한 인터페이스다. ACES II 는 원래 `xjoda`, `xvscf`, `xvcc` 처럼 모듈마다 독립 실행 파일을 만들고, 드라이버가 이를 차례로 실행하며 디스크 파일(JOBARC, MOINTS 등)로 상태를 주고받는다. pyaces 는 이 모듈들을 **하나의 공유 라이브러리** `aces2py.cpython-312-x86_64-linux-gnu.so` 로 링크하고, numpy 의 f2py 로 만든 Python 래퍼를 붙인다.

**Python** — `import aces2py; aces2py.Init(); aces2py.Runcc(0); aces2py.Dgetrecpy("TOTENERG", e)`

▼ f2py C 래퍼 (`Main/gen/aces2pymodule.c`)

**pyaces Fortran 셈(shim)** — `Main/init.F`, `a2_pyf/run*.F`, `a2_pyf/*recpy.F`, `*lstpy.F`, `aux_pyaces/*.F`

▼ 정적 링크 (`-Wl,--start-group` … 약 107개 아카이브)

**aces2 아카이브** — `aces2/lib/lib*.a` (joda, vmol, vscf, vtran, intprc, vcc, lambda, vee, props, intgrt, dirmp2 …) + 일부는 `build/patched_libs` 의 수정본

▼

**Intel MKL ILP64 (sequential)** · Intel Fortran 런타임

## 1.1 핵심 사실

- **aces2 소스는 건드리지 않는다.** pyaces 는 aces2 가 만든 `.a` 를 그대로 링크하고, 충돌이 나는 아카이브만 임시 복사본을 만들어 `objcopy` 로 심볼 이름을 바꾼다. aces2 트리는 고전 드라이버와 테스트 스위트의 공용 소스이기 때문이다.
- **모든 정수는 64비트**다. aces2 와 pyaces 는 `-i8` 로 컴파일되고 MKL 도 ILP64 로 링크한다.
- **한 프로세스에서 한 계산만** 한다. aces2 의 COMMON 블록 상태는 한 번 쓰는 것을 전제로 하며, 같은 프로세스에서 Vtran/Vcc 를 두 번 부르면 오류 없이 틀린 답으로 수렴한 사례가 있다.
- **가장 큰 위험은 링크 단계**다. 고전 빌드에서는 실행 파일마다 따로 링크되던 동명 루틴(서로 다른 구현)이 하나의 .so 안에서 충돌한다. 링커는 경고 없이 하나를 고르므로 결과가 조용히 틀릴 수 있다. 이를 다루는 장치가 5–6장의 핵심이다.
- **테스트는 aces2 fast 스위트를 재사용**한다. 125개 케이스 중 pyaces 로 실행 가능한 51개(SCF 9, CC 31, EOM 8, KS 3)를 돌려 JOBARC 에너지를 기준값과 $1\times10^{-6}$ $E_h$ 이내로 비교한다.

## 1.2 이 문서를 읽는 순서

처음이라면 2장(어떤 파일이 살아 있는지) → 4장(빌드) → 7–8장(API 와 실행) 순서로 읽는다. 새 Fortran 루틴을 노출하거나 aces2 모듈을 추가로 링크하려면 5–6장과 10장을 반드시 읽는다. 11장은 지금 코드에 남아 있는 결함 목록이다.

# 2. 디렉터리 구조

`pyaces/` 에는 2023년 원본 acespy 의 흔적이 많이 남아 있어서, 지금 빌드가 실제로 쓰는 파일과 그렇지 않은 파일을 구분하는 것이 중요하다. 사용 은 현재 빌드/테스트가 쓰는 것, 생성 은 빌드 산출물(gitignore), 레거시 는 쓰지 않는 것이다.

| 경로 | 상태 | 내용 |
|---|---|---|
| `build.sh` | 사용 | 빌드 전체(약 506줄). f2py 래퍼 생성 → C/Fortran 컴파일 → 패치 아카이브 → 링크 → import 시험 → 링크 감사. 충돌 해결 이력이 주석으로 기록되어 있다. |
| `Main/aces2.pyf` | 사용 | f2py 시그니처 파일. 모듈 이름 `aces2py` 를 정한다. **단, 최종 시그니처는 Fortran 소스에서 결정된다**(7.1절). |
| `Main/init.F`, `Main/aces2_main.F` | 사용 | `Init()` 진입점. Joda 로 ZMAT 를 읽고 `aces_init` 으로 코어 메모리·플래그·리스트를 초기화한다. |
| `Main/*.com` | 사용 | init.F 가 따옴표 `#include` 로 읽는 헤더 사본( `integer*8` 명시판). aces2 의 헤더와 내용이 다르다(11장). |
| `a2_pyf/*.F` | 사용 | Python 에 노출되는 래퍼와 드라이버: `run*.F`, `*recpy.F`, `*lstpy.F`, `*allpy.F`, `get1ehpy/get2ehpy.F`, `acesja*.F`, 그리고 내부용 `a2_get*/a2_put*.F`. `rungrads`, `run1props`, `run` 은 빌드에서 제외된다. |
| `aux_pyaces/*.F` | 사용 | `init_legacies` (레거시 COMMON 초기화), `get1ents` / `get2eints` (IIII 파일에서 AO 적분 읽기), `sym_expand`. |
| `tools/collision_audit.py` | 사용 | 링크 결과 감사(6장). `collision_baseline.txt` (296), `common_shadow_baseline.txt` (0). |
| `tools/compat_bin/ifort` | 사용 | `exec ifx "$@"` 두 줄짜리 셈. ifort 가 없는 oneAPI 에서도 `ifort` 이름으로 ifx 를 부른다. |
| `pyaces_run_case.py` | 사용 | 케이스 하나를 독립 프로세스에서 실행하고 결과를 표식 줄로 출력한다(8장). |
| `pyaces_suite_runner.py` | 사용 | 51개 케이스를 돌리고 PASS/MISMATCH/… 를 판정한다(9장). |
| `classify_suite.py`, `suite_classification.tsv` | 사용 | aces2 fast 스위트 케이스를 범주·진입점·기준값으로 분류한 표와, 그 표의 최초 생성기. **TSV 에는 손으로 고친 행이 있다**(9.4절). |
| `Main/gen/` | 생성 | f2py 산출물 `aces2pymodule.c`, `aces2py-f2pywrappers.f`. |
| `Main/build/` | 생성 | 모든 `.o` 와 최종 `aces2py.cpython-312-x86_64-linux-gnu.so`. |
| `build/patched_libs/` | 생성 | 심볼을 고친 aces2 아카이브 사본(약 37개). 빌드마다 지우고 다시 만든다. |
| `f2py_gen.log` | 생성 | f2py 래퍼 생성 로그. 래퍼가 어떻게 만들어졌는지 확인할 때 본다. |
| `Fxx/`, `include/`, `a2_pyf/include_no/`, `Main/Archive/`, `a2_pyf/Archive/`, `Main/Pilot_tests/`, `Exp/`, `Python/`, `rename.py`, `README`, `frequently_asked_questions` | 레거시 | 원본 acespy 시절의 공유 라이브러리 빌드, 옛 헤더, 시험 코드, f2py 튜토리얼. 현재 빌드와 무관하다(부록 A). |

> **소문자 `.f` 파일** — `a2_pyf/*.f`, `Main/init.f` 등은 예전 전처리 결과물이다. 빌드는 `*.F` 만 쓰므로 `.F` 를 고쳐야 한다.

# 3. 빌드 환경

## 3.1 필요한 도구

| 항목 | 요구 사항 |
|---|---|
| Fortran | Intel `ifx` (oneAPI 2023.2 · 2024.0 에서 검증). `ifort` 이름은 `tools/compat_bin` 셈이 해결한다. |
| C | `mpicc` (f2py C 래퍼와 `fortranobject.c` 컴파일용. 로컬에서는 gcc 를 감싼다) |
| Python | **3.12** + numpy (f2py 포함). 출력 파일 이름이 `cpython-312` 로 고정되어 있어 다른 버전에서는 import 되지 않는다. |
| BLAS/LAPACK | Intel MKL, `MKLROOT` 설정 필수. ILP64 · sequential 로 링크. |
| binutils | `ar`, `nm`, `objcopy` |
| 환경 모듈 | `module` 명령. `pyaces/build.sh` 가 스스로 `module purge` 후 다시 load 한다. |
| 선행 빌드 | aces2 가 먼저 빌드되어 `aces2/lib/lib*.a` 가 있어야 한다. |

## 3.2 configure 와 모듈 세트

최상위 `./configure` 가 툴체인을 찾아 `build_config.sh` 를 쓴다. 여기 기록된 `ACES_PROGRAM_MODULES` 를 최상위 `build.sh` 가 export 하고, `pyaces/build.sh` 는 `module purge` 뒤 이 세트를 다시 load 한다. 값이 없으면 HiPerGator 세트 (`intel/2025.1.0 openmpi/5.0.7 python/3.12`)로 대체된다.

```
# ccsd 서버
./configure --modules="intel/oneapi_2023.2.0 python/3.12.3" --np=4
# 로컬 워크스테이션: build_config.sh 의 ACES_PROGRAM_MODULES="intel-ompi/2024.0"
```

> **주의** — 모듈 세트에 Python 모듈을 꼭 넣는다. `module purge` 가 numpy 가 있는 Python 을 내려 버리면 f2py 단계가 실패한다.

## 3.3 로케일

`pyaces/build.sh` 는 `export LC_ALL=C` 를 강제한다(21행). 이 스크립트 안의 glob 과 `sort` 가 아카이브 멤버 순서와 `-l` 순서를 정하고, 그 순서가 동명 루틴 중 어느 것이 링크되는지를 바꾸기 때문이다. aces2 makefile 의 소스 목록도 같은 이유로 `$(sort …)` 로 감싸 두었다 (GNU Make 4.2 는 wildcard 를 정렬하지 않고, 4.3 은 로케일 순서로 정렬한다).

## 3.4 빌드 명령

```
./build.sh aces2 pyaces          # 최상위: aces2 다음 pyaces
./build.sh pyaces                # aces2 가 이미 있으면 pyaces 만
bash pyaces/build.sh             # 직접 실행 (ACES_PROGRAM_MODULES 를 지정할 것)
```

로그는 `build_logs/unified_build_pyaces.log`. 로컬 워크스테이션에서는 할당 코어 안에서 돌린다: `taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./build.sh pyaces`. ccsd 서버에서는 PBS(`build_logs/build_ccsd.pbs`)로만 빌드한다.

> **컴파일 플래그를 바꿨다면** — make 는 플래그 변경을 추적하지 않는다. aces2 플래그를 바꾼 뒤에는 `git clean -fdXq aces2` 로 산출물을 지우고 다시 빌드한다. pyaces 는 매번 `gen/` 과 `build/` 를 지우고 새로 만든다.

# 4. 빌드 파이프라인 (`build.sh`)

## 4.0 준비 (1–38행)

- `tools/compat_bin` 을 PATH 앞에 넣어 `ifort → ifx`.
- `module purge` → `module load $ACES_PROGRAM_MODULES`. 실패하면 configure 를 다시 하라는 안내와 함께 종료.
- `MKLROOT` 가 비어 있으면 종료. `MKL=$MKLROOT/lib/intel64` (없으면 `$MKLROOT/lib`).
- `export LC_ALL=C`.
- `NUMPY_INC`, `PY_INC`, `F2PY_SRC` (numpy 안의 `f2py/src`)를 지금 쓰는 python3 에 물어 정한다.
- `cd Main; rm -rf gen build; mkdir -p build`.

## 4.1 소스 목록

**`PYF_SOURCES`** (47–52행): `.pyf` 에 선언된 루틴을 *정의하는* 소스만 나열한다. f2py 에 `.pyf` 만 주면 빈 껍데기 래퍼가 생기고, 반대로 모든 소스를 주면 f2py 가 그 안의 모든 서브루틴을 자동으로 래핑하면서 대문자 F77 이름에서 대소문자 맹글링 오류가 난다. 그래서 필요한 파일만 고른다.

**`SOURCES`** (54–64행): 컴파일할 전체 Fortran. `a2_pyf/*.F` 중 `rungrads·run1props·run` 을 뺀 것, `Main/init.F`, `Main/aces2_main.F`, `aux_pyaces/*.F`. 따라서 `a2_get*/a2_put*`, `getlstpy_dbg`, `get1epy`, `returnpy` 는 링크되지만 Python 에는 노출되지 않는다.

## 4.2 단계별 동작

| 단계 | 명령 / 동작 | 산출물 |
|---|---|---|
| 1. 래퍼 생성 | `python3 -m numpy.f2py aces2.pyf $PYF_SOURCES -m aces2py --build-dir gen` (생성만, `-c` 없음). 이어서 `sed` 로 `F_FUNC(Name,…)` 의 첫 인자를 소문자로 바꾼다 — ifx 는 `init_` 처럼 소문자 심볼을 내보내는데 f2py 는 소스 대소문자 `Init` 을 쓰기 때문이다. | `gen/aces2pymodule.c` `gen/aces2py-f2pywrappers.f` `f2py_gen.log` |
| 2. C 컴파일 | `mpicc -DNDEBUG -fPIC -O2 -I. -I$NUMPY_INC -I$PY_INC -I$F2PY_SRC` 로 모듈 C 파일과 `fortranobject.c` | `build/aces2pymodule.o`, `build/fortranobject.o` |
| 3. Fortran 컴파일 | 아래 `FFLAGS` 로 `$SOURCES` 와 f2py 래퍼를 컴파일. 그 다음 `objcopy --redefine-sym props_=pyaces_vprops_props_ build/runee.o` (Runee 가 이름이 바뀐 vprops 의 PROPS 를 부르도록). | `build/*.o` |
| 4. 링크 | LIBDIRS/LIBNAMES 생성, 충돌 모듈의 패치 아카이브 생성(5장), 그리고 `ifx -shared -nofor-main`. | `build/aces2py.cpython-312-x86_64-linux-gnu.so` |
| 5. import 시험 | `python3 -c "import aces2py; print('IMPORT OK')"` — 실패하면 `set -e` 로 빌드 실패. | — |
| 6. 링크 감사 | `python3 tools/collision_audit.py \|\| true` — 권고용이며 빌드를 실패시키지 않는다. | 로그의 `@COLLISION_AUDIT:` 줄 |

### Fortran 플래그

```
FFLAGS="-diag-disable=10448 -i8 -fPIC -fp-model strict -O1 -assume minus0 -qopenmp \
        -D__fortran -D__fortran77 -D_INTEL -D_RECL_IS_WORDS_ \
        -I$ACES/include -I$ACES/acescore/include -I. -FI"
```

- `-i8`: 기본 INTEGER 를 8바이트로. aces2 와 MKL ILP64 에 맞춘다.
- `-FI`: 고정 형식 소스. `-D__fortran77` 은 aces2 헤더의 F77 분기를 고른다.
- 헤더는 aces2 의 `include`, `acescore/include` 를 쓴다(`pyaces/include/` 는 쓰지 않는다).

### 링크 명령

```
ifx -shared -nofor-main -o build/aces2py.cpython-312-x86_64-linux-gnu.so build/*.o \
    -L$PATCHED_LIBDIR $LIBDIRS -Wl,--start-group $LIBNAMES -Wl,--end-group \
    -L$MKL -Wl,--start-group -lmkl_intel_ilp64 -lmkl_lapack95_ilp64 \
           -lmkl_sequential -lmkl_core -Wl,--end-group -lpthread
```

- `LIBDIRS` / `LIBNAMES`: `find $ACES -maxdepth 2 -iname "*.a"` 로 매번 새로 만든다(정렬·중복 제거). aces2 의 모든 아카이브가 한 그룹 안에 들어가므로 순환 의존도 풀린다.
- `-L$PATCHED_LIBDIR` 가 맨 앞이라, 같은 이름의 패치본이 원본보다 먼저 선택된다.
- `-nofor-main`: ifx 가 자체 main 을 요구하지 않게 한다.

# 5. 하나의 .so 로 링크하기 — 심볼 충돌과 해결

고전 ACES II 빌드에서는 모듈마다 실행 파일을 따로 링크한다. 그래서 `vcc` 의 `FINISH` 와 `lambda` 의 `FINISH` 처럼 **이름은 같고 구현은 다른 루틴**이 있어도 문제가 없었다. pyaces 는 이것들을 한 .so 에 모두 넣으므로, 링커가 그중 하나를 골라 모든 호출자에게 준다. 링커는 경고하지 않는다. 결과는 세그폴트, `@GETLST: List (x,y) does not exist`, 혹은 **조용히 틀린 에너지**다. pyaces 의 링크 단계에는 이를 막기 위한 장치가 네 가지 있다.

## 5.1 MAIN__ 지역화

12개 모듈(`hbar joda lambda pccd props vcc vee vmol vmol2ja vscf vtran intprc`, `MAIN_COLLISION_MODULES`)은 Fortran PROGRAM 을 가지고 있어 각자 `MAIN__` 를 정의한다. 독립 실행 파일에는 필요하지만 한 .so 에서는 "multiple definition of MAIN__" 가 된다. 빌드는 이 모듈들의 아카이브를 임시 디렉터리에 `ar x` 로 풀고, `MAIN__` 를 정의한 오브젝트에 `objcopy --localize-symbol=MAIN__` 를 적용한 뒤 `build/patched_libs/lib<mod>.a` 로 다시 묶는다. aces2 의 원본 `.a` 는 수정하지 않는다.

## 5.2 `EXTRA_LOCALIZE_SYMBOLS` — 진 쪽 이름 바꾸기

`module:symbol` 쌍의 목록이다(411행 부근). 각 쌍에 대해 그 모듈 아카이브의 **모든 오브젝트**에 `objcopy --redefine-sym sym=pyaces_<mod>_sym` 을 적용한다. 정의와 그 모듈 내부의 호출이 함께 바뀌므로, 진 쪽 모듈은 자기 구현을 새 이름으로 계속 쓰고, 원래 이름은 이긴 쪽 구현에 남는다.

> **규칙** — 쌍이 효과를 내려면 그 모듈이 `MAIN_COLLISION_MODULES` 또는 `EXTRA_LOCALIZE_MODULES` 에 있어야 한다. 처리 대상이 아닌 모듈의 쌍은 **아무 경고 없이 무시**되고, `objcopy` 오류도 `2>/dev/null || true` 로 버려진다.

예전에는 `--localize-symbol` 을 썼으나, 그러면 진 쪽 모듈 자신의 호출까지 이긴 쪽으로 가 버려(lambda 의 `vlamcc` 가 vcc 의 FINISH 를 부름) 2026-08-11 에 `--redefine-sym` 로 바꿨다.

### 지금까지 해결한 주요 충돌

| 심볼 | 이름을 바꾼(진) 쪽 | 증상 / 비고 |
|---|---|---|
| `FINISH` | lambda, molcas | Runcc 가 수렴 에너지 대신 0회차 TOTENERG 를 보고 |
| `GSCHMIDT` | joda, libr, vcceh, vea | 인자 수 불일치로 SIGSEGV |
| `MODF` | lambda, fsip, vcceh, hcmult, vea, lcct | EOM 경로 오류 |
| `EXPDEN`, `PROPS` 외 vprops 보조 루틴 | props, vcceh / vprops | Runee 의 DIPOLE 계산 |
| `HBARXC`, `DT2INT2`, `DRAOLAD`, `T2TOAO`, `GETAOINF`, `Z2TOMO` | vcceh, lcct, hcmult, lambda, vee, dens | GETLST 리스트 없음, Z2TOMO 는 약 1e-5 오차 |
| `W45T3DRV` | lambda | lambda 의 8인자판이 trp 의 4인자판을 이김 |
| `CMPENG` | rpa | 단일 항이 빠져 2.49e-05 조용한 오차 |
| `XGEMM` | tools | DGEMM 잘못된 인자 |
| `CHECK_T2`, `INITIN/INITSN/DIISLST/DODIIS0` | ringcc / lambda | vcc 가 남의 DIIS·초기화 루틴을 사용 |

| | | |
|---|---|---|
| `ABCD` (COMMON 그림자) | intgrt | 모든 KS 케이스가 명령어 인출 중 폴트(5.4절) |
| `getdt1`, `fndmaxs`, `t2l2rng`, `r1l2y2c` | vcc, vee, lcct, lcct | 링크 순서에 따라 승자가 바뀌던 것(make 4.2 vs 4.3). eab3c4e6 에서 고정 |

각 항목의 자세한 원인과 진단 과정은 `build.sh` 의 주석(약 118–410행)에 기록되어 있다.

## 5.3 `NEWLY_LINKED_MODULES` — 새로 들어온 모듈 일괄 처리

`a2mix aux rpa scrnc` 는 원래 `.skip` 디렉터리였다가 다시 활성화된 모듈로, 고전 빌드에서는 다른 모듈과 함께 링크된 적이 없다. 빌드는 `aces2/lib/lib*.a` 전체의 `T` 심볼을 모아, 이 네 모듈이 다른 모듈과 공유하는 모든 심볼(`MAIN__` 제외)을 자동으로 `EXTRA_LOCALIZE_SYMBOLS` 에 추가한다. 로그에 `shadowed by <mod>, renaming: …` 로 찍힌다.

## 5.4 COMMON 블록이 서브루틴을 가리는 경우

Fortran `COMMON /abcd/` 는 `abcd_` 라는 tentative 심볼(`nm` 의 `C`)을 만든다. 서브루틴 `ABCD` 도 `abcd_` 다. 링커가 서브루틴이 들어 있는 오브젝트를 꺼내기 전에 COMMON 이 참조를 채워 버리면, `call abcd` 는 0으로 채워진 BSS 로 점프한다. 증상은 명령어 인출 중 SIGSEGV 이며, gdb 에서 `si_addr == $pc` 로 알아볼 수 있다.

- **rename-definer**: 정의한 모듈만 그 루틴을 부르면 `definer:sym` 을 `EXTRA_LOCALIZE_SYMBOLS` 에 넣는다(예: `intgrt:abcd_`).
- **rename-commons**: 다른 모듈에서도 부르면 이름을 바꿀 수 없으므로 COMMON 쪽을 옮겨야 한다.

잠재 위험 9개(mopac `deriv/grid/local/polar`, nddo `guess/local`, aces2·ccd_light `energy`, pdens `one/tab`, vdint `sym2`)는 미리 이름을 바꿔 두었다.

## 5.5 patched_libs 위생

`build/patched_libs` 는 `-L` 로 aces2/lib 보다 앞에 오므로 남아 있는 옛 파일이 진짜 아카이브를 가린다. 예전 실험에서 남은 `libintgrt.a` 때문에 `undefined symbol: intgrt_run_` 가 난 적이 있어, 지금은 빌드마다 `rm -rf` 후 다시 만든다.

## 5.6 어느 쪽이 이겼는지 확인하기

```
# 1순위: 링커에게 직접 묻는다 (링크 명령에 추가)
-Wl,-trace-symbol=getdt1_
# 2순위: .so 의 크기와 아카이브 크기를 맞춰 본다
nm -S Main/build/aces2py*.so | grep ' getdt1_$'
nm -S ../aces2/lib/libvcc.a ../aces2/lib/libvee.a | grep ' getdt1_$'
# COMMON 그림자 확인: T 가 아니라 B/D 이면 코드가 아니다
nm Main/build/aces2py*.so | grep -E ' (abcd_|hbar_)$'
```

# 6. 링크 결과 감사 도구 (`collision_audit.py`)

빌드 6단계에서 자동으로 실행되며, 직접 실행할 수도 있다. 이미 빌드된 `Main/build/aces2py*.so` 가 필요하다.

```
python3 tools/collision_audit.py            # 기준선과 비교해 보고
python3 tools/collision_audit.py --update   # 현재 결과로 두 기준선 파일을 덮어씀
```

## 6.1 무엇을 검사하나

**(1) 해석 차이** — 여러 모듈이 정의한 심볼마다:

1. `aces2/Makefiles/GNUmakefile` 의 `ACES_LINK_LISTS` 로 고전 링크 순서를 재구성한다.
2. .so 에 실제로 들어간 구현을 `nm -S` 크기로 판별한다(크기가 같아 구분이 안 되면 건너뜀).
3. 각 호출 모듈이 고전 빌드에서 받았을 구현(자기 것, 없으면 링크 목록에서 처음 정의한 라이브러리)과 비교한다.
4. 둘이 다르고, 주석을 뺀 소스 코드도 다르면 `symbol winner classic caller` 로 보고한다.

`build.sh` 의 `EXTRA_LOCALIZE_SYMBOLS` 와 `NEWLY_LINKED_MODULES` 를 직접 읽어 이미 이름을 바꾼 쌍은 제외한다.

**(2) COMMON 그림자** — 한 아카이브에서 `T`, 다른 아카이브에서 `C` 이고 .so 에서 B/b/D/d 인 심볼을 찾아 `rename-definer` 또는 `rename-commons` 판정과 함께 보고한다.

## 6.2 출력 읽기

```
@COLLISION_AUDIT: 0 subroutines shadowed by a same-named COMMON block ...
@COLLISION_AUDIT: 296 resolution differences (296 in the baseline)
  resolved since the baseline: ...              # 기준선에 있었는데 사라진 것
@COLLISION_AUDIT: NEW resolution differences ...
    getdt1_   .so gives vcc, lcct expects lcct   # 새로 생긴 것 — 조치 필요
```

## 6.3 NEW 항목이 나왔을 때

1. 두 구현을 비교한다(인자 수, 리스트 번호, 계산 내용). 같은 계산이면 기준선에 받아들여도 된다.
2. 다르면 이름을 바꿀 쪽을 정해 `"<loser>:<symbol>"` 을 `EXTRA_LOCALIZE_SYMBOLS` 에 추가한다. 그 모듈이 `EXTRA_LOCALIZE_MODULES` 에 있는지 확인한다.
3. pyaces 를 다시 빌드해 NEW 가 사라졌는지, 다른 심볼이 뒤집히지 않았는지 확인한다(모듈을 처리 대상에 추가하면 그 모듈의 다른 심볼 해석도 바뀔 수 있다).
4. 스위트를 돌린 뒤에만 `--update` 로 기준선을 갱신한다.

> **기준선은 작업 목록이 아니다** — 기준선의 296개는 "알고 받아들인" 차이다. 한 번에 "안전해 보이는" 42개를 고쳤다가 스위트가 45 → 31 PASS 로 떨어진 적이 있다. 또 `ACES_LINK_LISTS` 는 실행 파일을 만드는 모듈만 다루므로, 라이브러리 모듈(libr, libr2, tools, acescore …) 안의 호출자는 감사가 보지 못한다.

# 7. `aces2py` 모듈 API

## 7.1 시그니처는 Fortran 소스가 정한다

f2py 는 `aces2.pyf` 와 `PYF_SOURCES` 를 모두 받으면 모듈을 두 번 만든다. 한 번은 `.pyf` 로부터(소문자 이름, COMMON 블록 포함), 한 번은 소스 스캔으로부터(소스 대소문자)이고, 같은 `aces2pymodule.c` 에 쓰이므로 **두 번째가 이긴다**( `f2py_gen.log` 로 확인 가능). 그 결과:

- Python 이름은 소스의 대소문자를 따른다: `Init` , `Dgetrecpy` , `IGetrecpy` (G 대문자).
- 인자 타입과 intent 는 소스에서 온다. `.pyf` 의 `intent(inout)` 등은 무시된다. 모든 `Run*` 은 `Iuhf` 인자를 받는다.
- `.pyf` 의 COMMON 블록( `/flags/` , `/jobarc/` , `/lists/` …)은 노출되지 않는다.
- 정확한 시그니처는 `print(aces2py.Runcc.__doc__)` 또는 `gen/aces2pymodule.c` 의 docstring 으로 확인한다.

## 7.2 배열 인자 규칙

모든 함수는 반환값이 없는 Fortran 서브루틴이다. 결과는 **호출자가 넘긴 numpy 배열에 쓰인다**. f2py 는 배열 인자를 "input" 으로 래핑하므로, dtype 이나 메모리 배치가 다르면 조용히 **사본**을 만들어 넘기고, Fortran 이 쓴 값은 사본과 함께 사라진다.

> **반드시** — 실수형은 `np.zeros(n, dtype=np.float64)` , 정수형은 `np.zeros(n, dtype=np.int64)` 처럼 C-연속 1차원 배열을 미리 만들어 넘긴다. 레코드 이름은 8자이며 공백으로 채운다( `"IFLAGS "` ).

## 7.3 계산 드라이버

`Iuhf` : 0 = RHF, 1 = UHF/ROHF (Fortran 쪽은 `Integer*8` , C 쪽은 `long long` ). 어떤 드라이버도 `aces_fin` 을 부르지 않으므로 계산 뒤에도 세션이 열려 있어 레코드를 읽을 수 있다.

| 함수 | 소스 | 수행 내용 |
|---|---|---|
| `Init()` | `Main/init.F` | `aces2_main` → `Joda_` (ZMAT 해석, JOBARC 생성), `Aces_ja_fin` , `aces_init` (코어 할당, 플래그·리스트 초기화, JOBARC 재개방). **Runee 이외의 드라이버 전에 먼저 부른다.** |
| `Runints(Iuhf)` | `runints.F` | `Ints_` (vmol AO 적분) + `V2ja` |
| `Runscf(Iuhf)` | `runscf.F` | IIII 파일이 없으면 적분부터, 그 다음 `Vscf` |
| `Runcc(Iuhf)` | `runcc.F` | SCF 가 안 되어 있으면( `SCFEVCA0` 레코드 없음) SCF 부터, 이어서 `aces_init_chemsys` , `Vtran` , `Init_legacies` , `Intproc` , `Vcc` . MBPT/CC 계열 전반. |
| `Runhbar(Iuhf)` , `Buildhbar()` | `runhbar.F` , `buildhbar.F` | Runcc 단계 + `Hbar_` . SCF 가 끝나 있으면( `SCFEVCA0` ) Vtran 부터, CC 가 끝나 있으면( `CC_DONE` ) Hbar 만 돈다. Buildhbar 는 `IFLAGS(11)` 에서 iuhf 를 읽는다. **Runcc 다음에 부르지 않는다** — SCF 분기로 들어가 Vtran 을 두 번 부르게 된다. |
| `Runprops(Iuhf)` | `runprops.F` | `Props_` |
| `Runpccd(Iuhf)` | `runpccd.F` | 자체 완결: Joda, Ints, aces_init, V2ja, Vscf, Vtran, Intproc, Vcc, Vlambda, Pccd |

| | | |
|---|---|---|
| `Runee(Iuhf)` | `runee.F` | 자체 완결 EOM-CC: Joda, aces_init, Ints, V2ja, vprops Props(이름 변경판), Vscf, Vtran, Intproc, Vcc, Vlambda, Vee. **`Init()` 을 먼저 부르지 않는다.** |
| `Rundirmp2(Iuhf)` | `rundirmp2.F` | 필요하면 SCF, 그 다음 `Dirmp2_run(0)`. Iuhf 는 전달되지 않는다. |
| `Runintgrt(Iuhf)` | `runintgrt.F` | 필요하면 SCF, 그 다음 `Intgrt_run` (KS/DFT 수치 적분, XC 에너지) |
| `Acesjafin()` / `Acesjainit()` | `acesjafin.F` / `acesjainit.F` | JOBARC 세션 닫기(플러시) / 다시 열기. 이미 열린 세션을 다시 열면 `aces_exit` 로 종료된다. |

## 7.4 JOBARC 레코드

| 시그니처 | 설명 |
|---|---|
| `Dgetrecpy(Name, Dest, [Length])` | float64 레코드를 `Dest` 로 읽는다. 예: `"TOTENERG"`, `"SCFENEG "` |
| `IGetrecpy(Name, IDest, [Length])` | int64 레코드 읽기. 예: `"IFLAGS "` (100), `"NBASTOT "` |
| `Dputrecpy(Name, Source, [Length])` | float64 레코드 쓰기 |
| `Iputrecpy(Name, Isource, [Length])` | int64 레코드 쓰기 |
| `Cputrecpy(Name, Length, Source)` | 문자 레코드 쓰기 |
| `Dest = Cgetrecpy(Name, Length)` | 문자 레코드 `Length` 바이트를 읽어 `bytes` (길이 1024, 공백으로 채움)로 돌려준다. 1024 를 넘는 길이는 `aces_exit`. |
| `Length = Getreclenpy(Name)` | 레코드 길이(정수 워드)를 돌려준다. 레코드가 없으면 `-1`. |

길이는 정수 워드 단위이며, `-i8` 이므로 정수 1개 = 실수 1개 = 8바이트다. `Length` 를 생략하면 배열 길이를 쓴다.

## 7.5 MO 리스트(MOIO)

| 시그니처 | 설명 |
|---|---|
| `Getlstpy(Column_Index, No_columns, Irrep, Left_index, Right_index, Destn, Ndim)` | `A2_getlst(Destn, Column_Index, No_columns, 1, Left_index, Right_index)`. 실제로는 **`Left_index` 가 irrep, `Right_index` 가 리스트 번호**이며 `Irrep`·`Ndim` 은 쓰이지 않는다. |
| `Putlstpy(Source, Column_Index, No_columns, Irrep, Left_index, Right_index, [Ndim])` | Getlstpy 의 쓰기판 |
| `Getallpy(Destn, Irrep, Right_index, [Ndim])` | 전체 대칭 `Irrep` 인 리스트 `Right_index` 의 모든 irrep 블록 읽기 |
| `Putallpy(Source, Irrep, Right_index, [Ndim])` | 쓰기판 |

리스트 번호 참고표의 초안이 `Python/moio_len.py` 에 있다(예: T2(Ij,Ab) = 46, T1(AI) = 901, W<AB||IJ> = 14). 이 파일은 문법상 올바른 Python 이 아니므로 참고용으로만 본다.

## 7.6 AO 적분

| 시그니처 | 설명 |
|---|---|
| `Get1ehpy(Name, Dest, Ndim)` | `"overlap"` · `"oneh"` · `"kinetic"` 을 IIII 파일에서 읽어 `Ndim×Ndim` (`Ndim = NBASTOT`) 전체 행렬로 펼친다. |

| | |
|---|---|
| `Get2ehpy(Name, Dest, Ndim)` | `"2elints"` : IIII/IJIJ/IIJJ/IJKL 에서 `Ndim`$^4$ 크기의 2전자 적분 전체를 펼친다. 메모리에 주의. |

# 8. 계산 실행하기

## 8.1 실행 환경

- **작업 디렉터리**에 `ZMAT` 과 `GENBAS` 가 있어야 한다. ACES 는 같은 디렉터리에 JOBARC, JAINDX, IIII/IJIJ/IIJJ/IJKL, MOINTS, MOABCD, MOL, NEWMOS 등을 쓴다. 메모리 크기는 ZMAT 키워드로 정해진다.
- **라이브러리**: Intel 런타임과 MKL 이 필요하다. 빌드 때와 같은 모듈 세트를 load 하거나 `LD_LIBRARY_PATH` 를 맞춘다. 그렇지 않으면 `libmkl_intel_ilp64.so.2: cannot open shared object file` 로 import 가 실패한다.
- **import 경로**: `pyaces/Main/build` 를 `sys.path` 나 `PYTHONPATH` 에 넣는다.

## 8.2 기본 사용 예

```
import os, sys, numpy as np
os.chdir("/path/to/workdir")                 # ZMAT, GENBAS 가 있는 곳
sys.path.insert(0, "/path/to/ACES_PROGRAM/pyaces/Main/build")
import aces2py

aces2py.Init()                               # Runee 일 때는 생략
aces2py.Runcc(0)                             # 0 = RHF, 1 = UHF/ROHF

e = np.zeros(1, dtype=np.float64)
aces2py.Dgetrecpy("TOTENERG", e)
print("E =", e[0])
```

## 8.3 레코드 읽고 고치기

```
iflags = np.zeros(100, dtype=np.int64)
aces2py.IGetrecpy("IFLAGS  ", iflags)
iflags[1] = 1                                # IFLAGS(2) = CALC
aces2py.Iputrecpy("IFLAGS  ", iflags)
aces2py.Acesjafin()                          # 외부 실행 파일이 JOBARC 를 읽기 전에 플러시
# ... subprocess 로 ../aces2/bin/xvtran 등 실행 ...
aces2py.Acesjainit()
```

## 8.4 지켜야 할 규칙

- **한 프로세스에 한 계산.** 여러 ZMAT 을 한 프로세스에서 연달아 돌리지 않는다. 테스트 러너도 케이스마다 새 프로세스를 띄운다.
- **같은 모듈을 두 번 부르지 않는다.** Vtran/Vcc 를 같은 프로세스에서 두 번 부르면 틀린 값으로 수렴한 사례가 있다.
- **Runee, Runpccd 는 자체적으로 Joda 를 부른다.** Runee 앞에는 `Init()` 을 부르지 않는다.
- **대상 밖 계산**: 기하 최적화, 진동수, 해석적 미분(gradient/Hessian), Brueckner, `HFSTABILITY=FOLLOW`, CCSD(TQ) 는 아직 pyaces 로 실행할 수 없다( `rungrads.F` 는 빌드에서 제외).

## 8.5 FNO 특수 처리 ( `pyaces_run_case.py` )

ZMAT 에 `FNO_KEEP=` 이 있으면, 고전 드라이버는 동결할 가상 궤도를 고르기 위해 예비 Vtran/Intproc/Vcc/Fno 를 한 번 더 돈다. pyaces 에서 이를 프로세스 안에서 하면 Vtran/Vcc 를 두 번 부르게 되므로, 러너는 다음 순서로 처리한다.

1. `Runscf` 실행
2. `IFLAGS` 2, 93, 83, 100 을 임시로 바꾸고, `NUMDROPA` 를 `FNOFREEZ` 로 옮김
3. `Acesjafin()` 후 독립 실행 파일 `../aces2/bin/{xvtran,xintprc,xvcc,xfno}` 를 subprocess 로 실행 (로그: `fno_prelim_<exe>.log`)
4. 플래그 복원, IIII 가 소모되었으면 `xvmol` / `xvmol2ja` 로 재생성
5. `Acesjainit()` 후 `Runcc`

## 8.6 `pyaces_run_case.py` 출력 규약

```
python3 -u pyaces_run_case.py <workdir> <entrypoint> <iuhf> <record>
# 진행: PYACES_CASE: ...
# 성공: PYACES_CASE_VALUE: <repr>   /   PYACES_CASE_RESULT: SUCCESS <%.10f>
# 실패(종료 코드 1): IMPORT_FAILED, <ENTRY>_FAILED, RECORD_ZERO_OR_MISSING, DGETRECPY_FAILED
```

레코드 값이 정확히 0.0 이면 "없음"으로 간주한다.

# 9. 테스트 스위트

## 9.1 실행

```
./run_tests.sh pyaces                                    # 최상위 (로그: build_logs/unified_te
st_pyaces.log)
cd pyaces && python3 pyaces_suite_runner.py              # 직접
cd pyaces && python3 pyaces_suite_runner.py 010a 117b    # 일부 케이스만
```

- 케이스 입력: `aces2/tests/fast/cases/<name>/ZMAT`, 기저: `aces2/tests/fast/GENBAS`. pyaces 전용 테스트 디렉터리는 없다.
- 작업 디렉터리: `$PYACES_RUN_DIR/<name>`. `run_tests.sh` 는 HiPerGator 가 아니면 `build_logs/pyaces_suite_run` 으로 정한다. 러너 자체 기본값은 HiPerGator 의 `/blue/...` 경로다.
- 케이스마다 새 디렉터리에 ZMAT 복사, GENBAS 심볼릭 링크 후 `pyaces_run_case.py` 를 300초 제한으로 실행. 로그는 `<dir>/run.log`, 요약은 `$RUN_DIR/summary.tsv`.
- 로컬 워크스테이션: `OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./run_tests.sh pyaces`. ccsd 서버: `qsub build_logs/test_ccsd.pbs`.

## 9.2 판정

| 상태 | 조건 |
|---|---|
| PASS | 종료 코드 0, \|계산값 − 기준값\| ≤ $1\times10^{-6}$ $E_h$ (`TOLERANCE`, 케이스별 예외 없음) |
| MISMATCH | 종료 코드 0 이지만 차이가 허용치 초과 |
| NO_RESULT | 종료 코드 0 이지만 `PYACES_CASE_RESULT: SUCCESS` 줄이 없음 |
| TIMEOUT | 300초 초과 |
| FAIL_CLEAN | 양수 종료 코드 + `@ROUTINE:` 배너, "Assertion failed", "did not converge", "forrtl: severe" 등 인식된 메시지 |
| CRASH | 시그널로 종료(음수 코드) 또는 인식되지 않는 양수 종료 |

마지막 결과(2026-10-14): 51/51 PASS, 최대 차이 $4.95\times10^{-8}$ (케이스 078).

## 9.3 `suite_classification.tsv`

열: `name`, `category`, `reason`, `entrypoint`, `iuhf`, `record` (비교할 JOBARC 레코드, `TOTENERG` 또는 `SCFENEG`), `ref_value` (Fortran E/D 형식). 러너는 category 가 SCF·CC·EOM·PCCD·KS 인 행만 실행한다.

| 범주 | 개수 | 진입점 / iuhf |
|---|---|---|
| CC | 31 | Runcc (iuhf 0: 12, 1: 16), Rundirmp2 (3) |
| EOM | 8 | Runee (0: 6, 1: 2) |
| SCF | 9 | Runscf (0: 6, 1: 3) |
| KS | 3 | Runintgrt (0) |
| PCCD | 0 | — |
| EXCLUDED | 59 | 기하 최적화, VIB, 미분, MRCC, Brueckner, HFSTABILITY=FOLLOW, CCSD(TQ) 등 |

| | | |
|---|---|---|
| SCRIPT | 15 | ZMAT 없이 스크립트로 돌아가는 케이스 |

## 9.4 `classify_suite.py` 를 다시 돌리지 말 것

`classify_suite.py` 는 `aces2/tests/fast/test_results` 와 각 ZMAT 를 읽어 TSV 를 처음 만든 스크립트다. 그 뒤 다음 항목은 **손으로** 고쳐졌으므로, 스크립트를 다시 돌리면 사라진다: KS → Runintgrt 행, 117a/b/c → Rundirmp2, 020(CCSD(TQ))·022 계열(Brueckner)·083·124d.fno(HFSTABILITY=FOLLOW) 제외. TSV 는 직접 편집하거나, 먼저 이 예외들을 `classify()` 에 옮긴 뒤 재생성한다.

> **롤업의 빈틈** — `run_tests.sh` 의 pyaces 요약은 `PASS:` , `FAIL_CLEAN:` , `CRASH:` , `NO_RESULT:` 만 센다. MISMATCH·TIMEOUT 은 TOTAL 에 잡히지 않으므로, 결과가 51 이 아니면 `unified_test_pyaces.log` 의 Summary 블록을 직접 확인한다.

# 10. 확장하기 — 새 진입점과 새 테스트

## 10.1 새 Fortran 루틴을 Python 에 노출하기

1. **aces2 쪽을 호출 가능하게 만든다.** 대상이 PROGRAM 이면 본문을 서브루틴으로 분리한다("wrapper-split" 패턴: `dirmp2_run`, `intgrt_run`, `joda_`, `hbar_`, `props_`). aces2 를 먼저 다시 빌드한다.
2. **`a2_pyf/<name>.F` 를 작성한다.**
   - 고정 형식, `#include` 는 1열에서 시작(oneAPI 2023.2 ifort 가 들여쓴 include 를 거부함).
   - 모든 인자를 `Integer*8` / `Double Precision` 으로 **명시**한다. 그냥 `Integer` 로 쓰면 f2py 가 C `int`(4바이트)로 넘긴다(11장).
   - 소스의 서브루틴 이름 대소문자가 그대로 Python 이름이 된다.
   - 코어는 `icore.com`/`istart.com` 의 `Icore(i0), Icrsiz` 를 쓴다(`runcc.F` 참고). `aces_fin` 은 부르지 않는다.
3. **`Main/aces2.pyf` 의 interface 에 블록을 추가**한다(문서화 목적. 실제 시그니처는 소스가 정한다).
4. **`build.sh` 의 `PYF_SOURCES` 에 추가**한다. 컴파일은 `a2_pyf/*.F` glob 이 자동으로 한다(제외 목록에 없는 한).
5. **충돌을 처리한다.**
   - 새로 끌려 들어오는 모듈이 PROGRAM 을 가지면 `MAIN_COLLISION_MODULES` 에 추가.
   - 동명 루틴이 있으면 `loser:sym_` 을 `EXTRA_LOCALIZE_SYMBOLS` 에, 그 모듈을 `EXTRA_LOCALIZE_MODULES` 에 추가.
   - pyaces 쪽 오브젝트가 이름이 바뀐 루틴을 직접 부르면, 3단계 뒤에 `objcopy --redefine-sym` 을 추가(`runee.o` 의 예).
   - `.skip` 모듈을 다시 켜면 `NEWLY_LINKED_MODULES` 에 추가.
6. **빌드하고 확인한다.** "IMPORT OK", `f2py_gen.log` 의 두 번째(source-case) 모듈 빌드에 새 래퍼가 있는지, `print(aces2py.<Name>.__doc__)`.
7. **감사 결과**의 NEW 항목을 6.3절 순서대로 처리한다.
8. **호출하는 이름이 코드인지** 확인한다: `nm Main/build/aces2py*.so | grep ' <sym>$'` 이 `T` 여야 한다. `B`/`D` 이면 COMMON 블록에 묶인 것이다.
9. **테스트 케이스를 추가**하고 스위트 전체를 돌린다.

## 10.2 새 테스트 케이스 추가

1. aces2 스위트에 케이스를 넣는다: `aces2/tests/fast/cases/<name>/ZMAT` 와 `aces2/tests/fast/test_results` 의 3줄 항목(`name src` / `RECORD 1` / 기준값).
2. `pyaces/suite_classification.tsv` 에 행을 직접 추가한다(category, entrypoint, iuhf, record, ref_value).
3. `PYACES_RUN_DIR=... python3 pyaces_suite_runner.py <name>` 로 확인하고, `<dir>/run.log` 와 `summary.tsv` 를 본다.
4. 전체 스위트(`./run_tests.sh aces2 pyaces`)가 그대로 통과하는지 확인한다.

## 10.3 변경 후 점검 목록

| 무엇을 바꿨나 | 확인할 것 |
|---|---|
| a2_pyf / Main 의 Fortran | pyaces 빌드, IMPORT OK, 스위트 51/51 |

| | |
|---|---|
| build.sh 의 이름 바꾸기 목록 | 감사 결과(NEW 없음, resolved 항목이 의도한 것인지), 스위트, 그 다음 `--update` |
| aces2 소스나 플래그 | `git clean -fdXq aces2` 후 aces2+pyaces 재빌드, aces2 스위트 125/125, pyaces 51/51, 감사 |
| 툴체인 / 다른 기계 | 위 전부 + 두 기계의 감사 결과가 같은지 |

# 11. 알려진 문제

아래는 현재 코드에서 확인된 결함과 함정이다. 지금 스위트가 지나가는 경로에는 영향이 없지만, 해당 기능을 쓰거나 확장할 때 걸린다.

> **2026-10-23 수정됨** — 이 매뉴얼을 쓰며 찾은 결함 셋은 고쳤다.
>
> - **Runhbar/Buildhbar 의 `Call Hbar`**: 심볼 `hbar_` 는 서브루틴이 아니라 lambda 의 `COMMON /HBAR/` (.so 에서 `B hbar_`)이어서, SCF·CC 가 끝난 상태의 분기가 BSS 로 점프할 수 있었다. `Call Hbar_` (= `hbar__`)로 고쳤다. 감사 도구는 `T hbar_` 가 어디에도 없어 이 경우를 잡지 못한다 — 10.1절 8단계의 `nm` 확인이 필요한 이유다.
> - **`Iuhf` 폭**: `run*.F` 의 `Integer Iuhf` 를 f2py 가 4바이트 C `int` 로 넘기고 `-i8` Fortran 은 8바이트를 읽었다. `Integer*8 Iuhf` 로 선언해 양쪽을 64비트로 맞췄다.
> - **`Getreclenpy`, `Cgetrecpy`**: `Cf2py intent(out)` 을 붙여 길이와 문자열을 반환하게 했다(7.4절).

| # | 문제 | 내용 / 권장 조치 |
|---|---|---|
| 1 | `Getlstpy` 인자 의미 | `Irrep`, `Ndim` 은 무시되고 `Left_index` 가 irrep 로 쓰인다. |
| 2 | `.pyf` 가 사실상 무시됨 | COMMON 블록 노출과 intent 지정이 적용되지 않는다(7.1절). |
| 3 | Python 버전 고정 | 출력 파일 이름이 `cpython-312` 로 고정. 다른 버전은 `sysconfig.get_config_var('EXT_SUFFIX')` 를 써야 한다. |
| 4 | 헤더 사본 불일치 | `Main/*.com` 9개가 aces2 의 헤더와 다르다(`integer*8` 명시, include guard 없음). `-i8` 덕분에 크기는 같아 지금은 무해하지만, aces2 헤더가 바뀌면 init.F 와 나머지가 서로 다른 COMMON 배치를 보게 된다. |
| 5 | Runpccd 와 Init | Runpccd 는 스스로 Joda 를 부르는데 러너는 그 전에 `Init()` 을 부른다. PCCD 케이스가 없어 시험되지 않았다. |
| 6 | 오류 무시 | `EXTRA_LOCALIZE_SYMBOLS` 의 오타나 처리 대상이 아닌 모듈은 경고 없이 무시된다. |
| 7 | 최상위 빌드의 ifort 검사 | 최상위 `build.sh` 가 pyaces 셈을 PATH 에 넣기 전에 `ifort` 를 요구하므로, ifx 만 있는 툴체인은 거부된다. |
| 8 | 모듈 시스템 전제 | `pyaces/build.sh` 는 항상 `module purge/load` 를 하므로 setvars/path 모드나 `module` 명령이 없는 호스트에서 실패한다. |
| 9 | 테스트 롤업 · 분류기 | 9.3–9.4절 참고. |
| 10 | `Get1ehpy` 대칭 블록 오프셋 | `get1ehpy.F:101` 의 `Index = Index + Nelms*(K-1)` 가 의심스럽다. 대칭이 있는 분자에서 결과를 검증한 뒤 쓴다. |

# 12. 개발 이력과 서버 배포

## 12.1 단계별 이력

| 시기 | 내용 |
|---|---|
| 2023-11 | 원본 acespy (Ajith Perera). 모듈별 공유 라이브러리( `Fxx/` )와 f2py 로 시작. |
| 2026-08-08 – 08-11 | ACES_PROGRAM 에 subtree 병합( `ff876bec` ). 하드코딩 경로 제거, aces2 아카이브에서 링크 목록 생성( `37d57822` ). 첫 충돌(FINISH) 해결로 13 → 30/58 PASS. EOM 경로 충돌(GSCHMIDT, MODF, Z2TOMO …) 해결, localize → redefine-sym 전환. |
| 2026-09-02 – 09-04 | 테스트 러너 이식성, W45T3DRV·CMPENG·XGEMM·CHECK_T2·INITIN 충돌 해결, NEWLY_LINKED_MODULES 일괄 이름 바꾸기, 감사 도구 도입, Rundirmp2·Runintgrt 진입점, 제외 케이스 정리. |
| 2026-09-04 – 09-11 | COMMON 그림자 발견(ABCD → KS 크래시), KS 범주 추가로 51/51 달성, COMMON 감사 추가, 잠재 그림자 9개 선제 처리. |
| 2026-10-14 | 두 번째 기계(ccsd) 이식: init.F include 1열( `73944a6d` ), aces2 `-assume dummy_aliases` ( `dbed9377` , ifx 2023.2 의 vgamma2 오컴파일), 소스 목록 정렬· `LC_ALL=C` ·추가 이름 바꾸기( `eab3c4e6` ). 두 기계 모두 51/51, 감사 296 동일. |
| 2026-10-23 | 이 매뉴얼 작성 중 찾은 결함 수정: Runhbar/Buildhbar 의 `Hbar` → `Hbar_` , `Iuhf` 를 `Integer*8` 로, `Getreclenpy` · `Cgetrecpy` 가 값을 반환하도록. 51/51 유지. |

## 12.2 ccsd 서버 배포

- 설치 위치 `ycpark@172.24.7.130:/home/ycpark/program/ACES` . 서버는 인터넷이 없으므로 워크스테이션에서 `git push ccsd master` 로 갱신한다(서버 저장소는 `receive.denyCurrentBranch=updateInstead` ).
- 툴체인: `intel/oneapi_2023.2.0` (ifx 2023.2, MKL 2023.2, Intel MPI 2021.10) + `python/3.12.3` .
- 빌드와 테스트는 PBS 로만: `build_logs/build_ccsd.pbs` ( `qsub -v TARGETS=pyaces …` ), `build_logs/test_ccsd.pbs` , `build_logs/rebuild_test.pbs` (aces2 clean 재빌드 + 테스트).
- `-march=core-avx2` 빌드이므로 ccsd50/51/52 어느 노드에서도 실행된다.

## 12.3 도구체인 차이에서 배운 것

- 컴파일러 버전이 바뀌면 세 스위트(aces2, aces3, pyaces)를 모두 다시 돌린다. ifx 2023.2 는 2024.0 이 통과시킨 코드를 두 군데서 다르게 처리했다.
- 링크 감사 결과를 기계 간에 비교한다. 같은 소스라도 make·로케일 차이로 다른 구현이 링크될 수 있었다.

# 부록 A. 레거시 파일

| 경로 | 정체 |
|---|---|
| `Fxx/` | 2023년 acespy 의 모듈별 공유 라이브러리 빌드용 aces2 소스 사본(45개 디렉터리, 약 6,100 파일). 현재 aces2 와 상당히 달라졌고 어떤 스크립트도 참조하지 않는다. |
| `include/`, `a2_pyf/include_no/` | 옛 헤더 사본. 빌드는 aces2 의 include 를 쓴다. |
| `Main/Archive/`, `Main/compile_acespy` | 초기 프로토타입과 옛 `f2py -c` 빌드 레시피. |
| `a2_pyf/Archive/` | 루틴별 `.pyf` 를 쓰던 시절의 파일과 시험 코드. |
| `Main/Pilot_tests/` | COMMON/icore 초기 시험. |
| `Exp/` | 독립 f2py 튜토리얼(`add.F`, `check_strngs.F` …). |
| `Python/moio_len.py` | MO 리스트 번호 참고표 초안(문법 오류 있음). |
| `rename.py` | `.a → .so` 이름 바꾸기(하드코딩 경로). |
| `README`, `frequently_asked_questions` | 옛 빌드 메모, 일반 f2py FAQ. |
| `suite_classification.tsv.before_*` | 분류 수정 전 TSV 스냅숏. |
| `LICENSE` | GPL v3 — **유효**. |

# 부록 B. 명령 요약

```
# 빌드 (로컬, 할당 코어 안에서)
taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./build.sh aces2 pyaces

# 테스트
OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 \
  taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./run_tests.sh aces2 pyaces
cd pyaces && python3 pyaces_suite_runner.py 010a       # 한 케이스

# 링크 감사
cd pyaces && python3 tools/collision_audit.py
cd pyaces && python3 tools/collision_audit.py --update # 검증 후에만

# 모듈 내용 확인
python3 -c "import sys; sys.path.insert(0,'pyaces/Main/build'); import aces2py; print(aces2
py.__doc__)"
nm pyaces/Main/build/aces2py*.so | grep ' runcc_$'

# aces2 플래그 변경 후 clean 재빌드
git clean -fdXq aces2 && ./build.sh aces2 pyaces

# ccsd 서버
git push ccsd master
ssh ycpark@172.24.7.130 'cd program/ACES/build_logs && qsub rebuild_test.pbs'
```